

Lab 15

To Detect Objects

Aim: To implement a YOLO model to detect objects

Objective :

- * To study how YOLO detects multiple objects in a single image.
- * To understand the architecture and working of a pre-trained YOLO model
- * To use transfer learning for custom object detections

Pseudo code

- 1) Install and import (YOLO package)
- 2) Load a pre-trained YOLO model
- 3) Load a test image or use a camera

Frame

- 4) Run the `model.predict()` method to detect objects
- 5) Display bounding boxes and class labels
- 6) Save the annotated output image.

Implement a YOLO model

27/10/2024 Lab 15 To Detect Objects

Aim: To implement a YOLO model to detect objects

Objective :

- * To study how YOLO detects multiple objects in a single image.
- * To understand the architecture and working a pre-trained YOLO model
- * To use transfer learning for custom object detections

Pseudo code

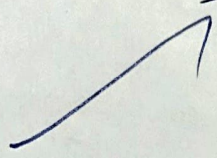
- 1) Install and import (YOLO package)
- 2) Load a pre-trained YOLO model
- 3) Load a test image or use a camera

Frame

- 4) Run the `model.predict()` method to detect objects
- 5) Display bounding boxes and class labels
- 6) Save the annotated output image.

Output

Image size: 720 x 1280 2 persons in the background
speed 89.9ms processed 22.9m inference
put Image at shape (1, 3, 384, 440)



Observation

- 1) The pre-trained YOLO v8s model successfully detected multiple objects such as cars, buses, person in single frame
- 2) Each object was enclosed in a bounding box with a class label and confidence score
- 3) The inference time per image was very low
- 4) The model demonstrated strong generalization without additional training
- 5) The visualization clearly showed YOLO's ability to detect overlapping objects in complex scenes

Result

Successfully implemented pre-trained

YOLO.

Best

```
print("\n Loading YOLOv5 model (this may take a few seconds)...")
model = torch.hub.load('ultralytics/yolov5', 'yolov5s', pretrained=True)

# Step 3: Perform inference on the image
img = Image.open(img_path)
results = model(img)

# Step 4: Print detection results
print("\n Detection Results")
results.print() # prints detected objects, confidence, coordinates

# Step 5: Display the image with bounding boxes
Follow link (ctrl + click)
results.show() # opens the image window with labels and boxes

# Step 6: (Optional) Save output image to a folder
output_folder = "yolo_output"
results.save(save_dir=output_folder)
print(f"\n Detection completed! Output saved in folder: '{output_folder}'")

# -----
```




#lab 15

```
import torch
from PIL import Image
import matplotlib.pyplot as plt
import urllib.request
import os

# Step 1: Automatically download a sample image
img_path = "street.jpg"
if not os.path.exists(img_path):
    url = "https://ultralytics.com/images/zidane.jpg" # sample image
    urllib.request.urlretrieve(url, img_path)
    print(" Sample image downloaded successfully: street.jpg")
else:
    print(" Image already exists:", img_path)

# Step 2: Load the pre-trained YOLOv5 model
# This will download YOLOv5 automatically from Ultralytics (first time only)
```



```
detected_img_path = os.path.join(output_folder, os.path.basename(img_path))
if os.path.exists(detected_img_path):
    detected_img = Image.open(detected_img_path)
    plt.imshow(detected_img)
    plt.axis('off')
    plt.title("YOLOv5 Object Detection Result")
    plt.show()
```

Image already exists: street.jpg

Loading YOLOv5 model (this may take a few seconds)...

Creating new Ultralytics Settings v0.0.6 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com>

Using cache found in /root/.cache/torch/hub/ultralytics_yolov5_master

YOLOv5 🚀 2025-11-1 Python-3.12.12 torch-2.8.0+cu126 CPU